# Variabili e tipi di dato

Lezione 3 — 1IOT, SST-ICT-01 — Bit, byte e i primi tre tipi di variabile

18/09/2026

## 1 Ripasso: bit, byte, e numero di combinazioni

**Definizione**

Un **bit** è la più piccola informazione manipolabile da un computer (0 o 1). Un **byte** è un gruppo di 8 bit. Una **word** è un gruppo di 2 byte, cioè 16 bit.

Con $n$ bit si possono rappresentare $2^n$ **combinazioni** diverse: con 8 bit, $2^8 = 256$ combinazioni; con 16 bit, $2^{16} = 65.536$ combinazioni.

## 2 Dove il computer tiene i dati: la RAM

Un programma, per funzionare, deve tenere a mente i **dati** su cui lavora – proprio come, per seguire una ricetta, dobbiamo ricordare le **quantità** degli ingredienti.

**Esempio**

Se ti chiedo di preparare la pasta e ti dico di mettere 100 grammi di farina, un uovo e 2 grammi e mezzo di sale, questi numeri (100, 1, 2,5) devi **tenerli a mente** da qualche parte – magari scritti su un pezzo di carta. Il computer, allo stesso modo, deve "**segnarsi**" da qualche parte i numeri con cui sta lavorando.

Il computer li tiene nella **RAM** (Random Access Memory).

**Definizione**

**Random Access Memory**: non significa "**memoria ad accesso casuale**" nel senso di **imprevedibile**, ma che si può **accedere** a *qualunque* **cella di memoria** direttamente, senza dover leggere tutte quelle precedenti (come accadeva con le vecchie memorie sequenziali). Questo rende la RAM molto più **veloce**.

**Esempio**

La RAM funziona come un **armadio con tanti cassetti**: ogni cassetto ha un nome (es. "calzini") e contiene un numero (es. 12). Senza un'**etichetta** che dica cosa contiene ogni cassetto, chiunque guardi solo il numero non saprebbe a cosa si riferisce – per questo un programma deve sempre **sapere cosa rappresenta** ogni dato che tiene in memoria.

# 3 I cassetti hanno dimensioni diverse: i tipi di dato

**Nota**

Un cassetto grande per contenere pochi oggetti piccoli è uno **spreco di spazio**; un cassetto piccolo non basta per oggetti grandi. Allo stesso modo, nel computer scegliamo la **dimensione giusta** del "cassetto" (variabile) in base a cosa dobbiamo memorizzare – le dimensioni sono sempre **multipli del byte**.

## 3.1 char: 1 byte

**Definizione**

Un **char** occupa 1 byte (8 bit) e può contenere numeri interi da **0 a 255** (256 combinazioni).

**Esempio**

Il **numero di allievi** di una classe (difficilmente sopra 255) può essere memorizzato in un **char**.

## 3.2 int: 2 byte (in questo contesto)

**Definizione**

Un **int** occupa (in questo contesto) 2 byte (16 bit). Contiene numeri interi **con segno**, da **-32.768 a +32.767**.

**Attenzione**

Perché il **range** non va da -32.768 a +32.768 in modo **simmetrico**? Perché lo **zero** occupa già una delle 65.536 combinazioni disponibili: se il range fosse perfettamente simmetrico, ci sarebbero **due zeri diversi** (+0 e -0), un problema per i calcoli. Per questo l'intervallo è leggermente asimmetrico: pensate di contare in sequenza $\ldots, -3, -2, -1, 0, \ldots$ – se si desse spazio simmetrico anche ai positivi, si otterrebbero un "**più zero**" e un "**meno zero**" distinti, che è un problema per i conti (lo zero **non ha segno**). Per questo si **toglie una combinazione** dal lato positivo: il range risultante è leggermente più esteso verso i negativi.

**Nota**

In realtà la **dimensione** di un `int` **non è fissa**: dipende dal ***parallelismo*** dell'**architettura** della macchina (quanti bit può gestire contemporaneamente). Su una macchina a 8 bit, l'int è comunque **almeno a 16 bit**; su un PC moderno a 64 bit, l'int può arrivare a 64 bit, rappresentando numeri **enormemente più grandi**. Il nostro **ambiente di programmazione online** lavora a **32 bit**.

### 3.3 float: 4 byte

**Definizione**

Un **float** occupa 4 byte (32 bit) e permette di memorizzare numeri **con la virgola**, in un **intervallo** di circa $\pm 2$ miliardi (per un totale di circa 4,3 miliardi di combinazioni).

**Attenzione**

Se si tenta di memorizzare un numero con la virgola (es. 12,99) in un char o in un int, **la parte decimale viene persa** (non **arrotondata**: semplicemente **troncata**). 12,99 memorizzato in un char diventerebbe 12, non 13. È una fonte comune di errori per chi comincia a programmare.

## 4 Scegliere il cassetto giusto

**Esempio**

Usare sempre `float` "per sicurezza" può sembrare comodo, ma spreca memoria: un programma che usa **più memoria del necessario** richiede più RAM per funzionare, rendendolo meno efficiente. Scegliere il **tipo di dato più piccolo** che soddisfa le proprie esigenze è una **buona pratica** di programmazione, non solo un dettaglio tecnico.

## 5 Cosa ricordare di questa lezione

- Un **bit** è l'**informazione minima** (0/1); un **byte** è un gruppo di 8 bit.
- La **RAM** permette l'**accesso diretto** a qualunque cella di memoria, senza leggere le precedenti.
- Una **variabile** è un "cassetto" nominato in memoria, di **dimensione adeguata** al dato che deve contenere.
- **char** (1 byte, 0-255), **int** (2+ byte, con segno, **dimensione dipendente dall'architettura**), **float** (4 byte, **numeri con la virgola**).
- Usare char/int per un numero con la virgola **tronca** la **parte decimale**, senza arrotondare.
- Scegliere il **tipo di dato più piccolo** adeguato al bisogno ottimizza l'uso della memoria.

**Esempio**

**Da fare per la prossima lezione**: quale **tipo di dato** useresti per memorizzare l'altezza di una persona in metri (es. 1,75)? E per il **numero di scarpe** che possiedi? Motiva la scelta.